

Chapter 15 - JPA Criteria API

1. What is the Criteria API?

The JPA Criteria API provides a type-safe, programmatic way to build queries in Java. It is useful for dynamic queries where conditions are decided at runtime.

HQL	Criteria API
String-based	Java API (type-safe)
Easy for static queries	Best for dynamic queries
Compile-time typo checks limited	Better IDE refactoring support

2. Main Components

CriteriaBuilder creates query objects and expressions. CriteriaQuery defines the result type. Root represents the entity being queried. Predicate represents conditions.

3. Query Flow

```
EntityManager
|
getCriteriaBuilder()
|
CriteriaBuilder
|
createQuery()
|
CriteriaQuery
|
Root<Employee>
|
Predicate(s)
|
TypedQuery
|
Database
```

4. Basic Example

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Employee> cq =
    cb.createQuery(Employee.class);

Root<Employee> root = cq.from(Employee.class);

cq.select(root);

List<Employee> list =
    em.createQuery(cq).getResultList();
```

5. WHERE Clause

Predicates are used to build conditions dynamically.

```
Predicate p =
    cb.equal(root.get("department"), "IT");

cq.where(p);
```

6. Sorting & Pagination

Use `orderBy()` for sorting and `setFirstResult()/setMaxResults()` for pagination.

7. Group By & Aggregate Functions

Criteria API supports `count()`, `avg()`, `min()`, `max()`, `sum()`, `groupBy()` and `having()` using `CriteriaBuilder` expressions.

8. CriteriaUpdate & CriteriaDelete

JPA 2.1 introduced `CriteriaUpdate` and `CriteriaDelete` for type-safe bulk update and delete operations.

9. Best Practices

- Prefer Criteria API for dynamic search screens.
- Use HQL or JPQL for simple static queries.
- Keep predicates modular for readability.
- Use typed queries to improve type safety.

10. Interview Questions

1. What is the Criteria API? 2. Difference between HQL and Criteria API? 3. What is `CriteriaBuilder`? 4. What is a Predicate? 5. When should you use Criteria API instead of HQL?